

Building AI Agents with Claude

From Hello World to Autonomous Production Systems

30 modules · 11 tracks · Generated 2026-05-10

TRACK 0 · OVERVIEW

M00

Course Overview & Agent Lifecycle

CORE IDEA

An agent is an LLM in a loop, with hands.

KEY PSEUDOCODE

```
// The universal agent pattern
DEFINE tools // e.g. search_db, send_email
DEFINE history = [user's question]

LOOP:
  reply = SEND history TO Claude WITH tools

  IF reply asks for a tool:
    result = RUN the requested tool
    APPEND reply AND result TO history
    CONTINUE loop

  ELSE:
    RETURN reply TO user
  BREAK loop
```

REMEMBER

✗ *"Agents run autonomously, 24/7."*

✓ An agent runs only when your code calls it. Between calls, nothing happens. It's a program with an LLM inside a loop — not a self-directed entity.

M01 LLM Mental Model**CORE IDEA**

LLMs Predict Tokens. They Don't Think.

KEY PSEUDOCODE

```
// One Claude API call, simplified
INPUT prompt, temperature, max_tokens

tokens = TOKENIZE(prompt)
output = []

REPEAT up to max_tokens times:
  scores = RUN MODEL(tokens + output)
  probs = SOFTMAX(scores / temperature)
  next_token = SAMPLE(probs)

  IF next_token == STOP: BREAK
  output.APPEND(next_token)

RETURN DETOKENIZE(output)
```

REMEMBER

✗ *"Claude reasons like a person."*

✓ It pattern-matches over language. There's no internal model of truth and no comprehension — which is exactly why it can produce confident, plausible, completely wrong answers.

M02 Tokens, Context & Cost**CORE IDEA**

What is a token, really?

KEY PSEUDOCODE

```
// PRE-FLIGHT CHECK before every Claude call

DEFINE CONTEXT_LIMIT = 200,000 // tokens
DEFINE RESERVE = 4,096 // for response

FUNCTION can_send(system, history, message):
  input_tokens = COUNT_TOKENS(system + history + message)
  available = CONTEXT_LIMIT - input_tokens

  IF available < RESERVE THEN
    RETURN "too big - summarize history first"
  ELSE
    RETURN "ok to send"

// AFTER the call, read the real numbers
response = CALL Claude(...)
```

REMEMBER

✗ *"One token equals one word."*

✓ Common words ("the", "is") are one token. Long or rare words split: "understanding" = "understand" + "ing" = 2 tokens. A space, a comma, an emoji — each is its own token too.

CORE IDEA

The Prompt Is the Whole Input

KEY PSEUDOCODE

```
DEFINE system_prompt =  
  <ROLE>senior code reviewer</ROLE>  
  <CONSTRAINTS>concise, fix-oriented</CONSTRAINTS>  
  <OUTPUT_FORMAT>bugs / perf / style</OUTPUT_FORMAT>  
  
DEFINE history = [] // user/assistant pairs  
  
ON each user_turn:  
  APPEND {role: USER, content: turn} TO history  
  SEND TO Claude:  
    system = system_prompt  
    messages = history  
  RECEIVE assistant_reply  
  APPEND {role: ASSISTANT, content: reply} TO history
```

REMEMBER

✗ *"Longer prompts always give better answers"*

✓ There is a sweet spot. A 50-word prompt is usually too vague, but a 5,000-word prompt wastes tokens and can confuse Claude with contradictory rules. Most production system prompts land in 200–800 words. Be specific, not verbose.

M04 Structured Output

CORE IDEA

Agents do not just generate text for humans — they produce data that your code consumes.

KEY PSEUDOCODE

```
// 1. DEFINE schema (the form)
DEFINE tool "extract_contact"
  FIELDS: name (str, required),
          email (str, required),
          phone (str, optional)

// 2. CALL Claude, force the tool
FOR attempt IN 1..MAX_RETRIES:
  response = SEND to Claude
  tools = [extract_contact]
  tool_choice = "extract_contact"
  messages = prompt + last_error

// 3. PARSE the tool_use block
```

REMEMBER

✗ *"Asking nicely for JSON is enough."*

✓ Prompt-only JSON fails 5–15% of the time in production. Tool use fails under 0.5% because Claude is specifically trained on the tool_use format.

M05 Function Calling

CORE IDEA

Claude proposes. Your code executes.

KEY PSEUDOCODE

```
# 1. DESCRIBE the tools available
DEFINE tool "get_weather"
  description: "Current weather for a city"
  input: city (required, string)

# 2. START the conversation
messages = [user_question]

WHILE iterations < MAX:
  response = CALL Claude(messages, tools)

  IF response.stop_reason == "end_turn":
    RETURN response.text # done

  IF response.stop_reason == "tool_use":
    FOR EACH tool_call IN response:
```

REMEMBER

✗ *"Claude executes the tool itself."*

✓ Claude only asks to call a tool. Your code reads the request, decides whether to honor it, and runs the function. Claude is the decision-maker; your application is the executor.

M06 Multi-Tool Orchestration

CORE IDEA

Run independent tools at once. Chain dependent ones. Fail loud, recover fast.

KEY PSEUDOCODE

```
# Filter tools to ones relevant for THIS request
tools = REGISTRY.pick_for(user_question)

messages = [user_question]
WHILE iterations < MAX:
    response = CALL Claude(messages, tools)

    IF response.stop_reason == "end_turn":
        RETURN response.text

# Claude may return MANY tool_use blocks at once
results = []
PARALLEL FOR EACH call IN response.tool_uses:
    TRY:
        out = RUN call.name(call.input)
```

REMEMBER

✗ *"More tools = more capable agent."*

✓ Tool-selection accuracy degrades past 5–6 tools. Each schema burns 200–500 prompt tokens and adds ambiguity. A focused 4-tool agent beats a sprawling 18-tool one almost every time.

M07 MCP — Model Context Protocol

CORE IDEA

USB-C for AI tools.

KEY PSEUDOCODE

```
# Create the server
server = NEW MCPServer(name="filesystem")

# 1. Tool: an action with side effects
DEFINE TOOL "write_file":
    input: path, content
    description: "Write text to a file at the given path"
    on_call(input):
        RETURN file_system.write(input.path, input.content)

# 2. Resource: read-only data
DEFINE RESOURCE "file-tree://cwd":
    on_read():
        RETURN list_directory_tree(".")
```

REMEMBER

✗ *"MCP is just another REST or GraphQL."*

✓ REST and GraphQL are general-purpose web standards. MCP is purpose-built for AI: it includes capability discovery, version negotiation, and the Resource / Tool / Prompt typology that maps cleanly to how models use context.

M08 Conversation Management

CORE IDEA

The API is stateless. Memory is your code.

KEY PSEUDOCODE

```

CLASS ConversationManager:
    messages = []
    system_prompt = "..."

    METHOD send(user_text):
        messages.append({user: user_text})

        # Trim if needed before calling
        IF token_count(messages) > budget:
            messages = trim_strategy(messages)

    TRY:
        reply = CALL Claude(system, messages)
        messages.append({assistant: reply})
    RETURN reply

```

REMEMBER

✗ *"Claude remembers our previous conversation."*

✓ No. The API has no server-side storage. If your messages array doesn't include a turn, it never happened. The "memory" is entirely your code's responsibility.

M09 RAG — Retrieval-Augmented Generation

CORE IDEA

Search first. Then generate.

KEY PSEUDOCODE

```

# INGEST (run once when docs change)
FOR EACH doc IN documents:
    chunks = SPLIT(doc, size=500, overlap=50)
    FOR EACH chunk IN chunks:
        vec = EMBED(chunk)
        vector_db.STORE(vec, chunk, source=doc.path)

# QUERY (every user question)
FUNCTION ask(question):
    q_vec = EMBED(question)
    top = vector_db.SEARCH(q_vec, k=3)

    context = JOIN(top, format="[Source N] {chunk}")
    prompt = "Answer using ONLY this context. "

```

REMEMBER

✗ *"RAG is just fine-tuning."*

✓ Fine-tuning rewrites model weights, costs \$5K–\$50K, and locks you to one snapshot of the data. RAG never touches the model — it just adds fresh text to the prompt at query time. Update your docs, RAG updates instantly.

M10 Advanced RAG Patterns

CORE IDEA

Pre-search. Search smarter. Re-score afterward.

KEY PSEUDOCODE

```
# 1. Optional: rewrite the query (HyDE)
hypo = CALL Claude("Write a doc paragraph that answers: " + Q)

# 2. Run two retrievers in parallel
vec_hits = vector_db.SEARCH(EMBED(hypo), k=20)
kw_hits = bm25.SEARCH(Q, k=20)

# 3. Fuse with Reciprocal Rank Fusion
scores = {}
FOR EACH list IN [vec_hits, kw_hits]:
  FOR rank, doc IN enumerate(list):
    scores[doc] += 1 / (60 + rank)
candidates = top_n(scores, 20)

# 4. Re-rank with Claude as cross-encoder
```

REMEMBER

✗ *"Hybrid search is always better than pure semantic search."*

✓ Not for every query. BM25 wins on exact identifiers, codes, and proper nouns. For abstract questions ("explain debtor risk factors") vector-only can outperform hybrid because BM25 just adds noise from common words.

M11 Multi-Layer Memory

CORE IDEA

One memory store doesn't scale. Layer them.

KEY PSEUDOCODE

```
CLASS WorkingMemory:
  state = {} # key → value, with timestamps

  METHOD set(key, value): state[key] = value
  METHOD get(key): RETURN state.get(key)
  METHOD clear(): state = {}

  METHOD to_prompt():
    RETURN "[Working memory: " + format(state) + "]"

# Each turn:
context = []
context.append(working.to_prompt())
context.append(episodic.search(user_q, k=3))
context.append(procedural.match(user_q)) # may be empty
```

REMEMBER

✗ *"Every agent needs all three memory tiers."*

✓ A simple FAQ bot needs zero. A single-session helper might only need working memory. Add tiers when a real problem demands them — not because the architecture diagram looks neat.

M12 ReAct Agent Loop**CORE IDEA**

One step is a chatbot. A loop is an agent.

KEY PSEUDOCODE

```
FUNCTION run_agent(user_question):
  messages = [{ role: "user", content: user_question }]
  iters = 0

  WHILE iters < MAX_ITERS:
    response = CALL Claude(system, tools, messages)
    messages.append({ role: "assistant", content: response.content })

    IF response.stop_reason == "end_turn":
      RETURN final_text(response) # done

    IF response.stop_reason == "tool_use":
      results = []
      FOR block IN response.content WHERE type == "tool_use":
        output = RUN tool(block.name, block.input)
```

REMEMBER

✗ *"ReAct needs a special framework like LangChain or CrewAI."*

✓ No. Claude's Messages API already produces ReAct: text blocks are Reason, tool_use blocks are Act, tool_result messages are Observe. You need a while loop and tool execution — that's it.

M13 Planning & Task Decomposition**CORE IDEA**

Plan before you ReAct.

KEY PSEUDOCODE

```
FUNCTION run_agent(user_request):
  intent = CALL Claude("classify:", user_request)
  IF intent.complexity == "low":
    RETURN CALL Claude(user_request) # skip planning

  plan = CALL Claude("decompose:", user_request)
  IF has_cycle(plan): plan = CALL Claude("replan:", plan)

  done = {}
  WHILE ready = [t FOR t IN plan IF deps_met(t, done)]:
    results = PARALLEL [execute(t) FOR t IN ready]
    FOR (t, r) IN results:
      IF r.failed: r = retry_once(t)
    done[t.id] = r # may be skipped on fail
```

REMEMBER

✗ *"Every request needs a plan."*

✓ No. "What's the weather?" doesn't need decomposition — the planning call adds latency and tokens for zero gain. That's why intent classification comes first: route simple requests straight to Claude, reserve planning for genuinely multi-step work.

CORE IDEA

Specialists beat the kitchen-sink generalist.

KEY PSEUDOCODE

```
DEFINE agent(name, system_prompt, tools):  
    RETURN {name, system_prompt, tools}  
  
researcher = agent("researcher", RESEARCH_PROMPT, [search])  
writer = agent("writer", WRITER_PROMPT, [])  
editor = agent("editor", EDITOR_PROMPT, [grammar])  
reviewer = agent("reviewer", REVIEWER_PROMPT, [])  
  
FUNCTION handoff(from_agent, to_agent, payload, goal):  
    RETURN { sender:from_agent, receiver:to_agent,  
            task_id, type:"task", payload,  
            original_goal:goal, instructions }  
  
FUNCTION orchestrate(user_goal):  
    notes = CALL researcher WITH handoff("sup", "researcher", {topic}, user_goal)
```

REMEMBER

✗ *"More agents = better results."*

✓ Each agent adds an extra Claude call and a place context can leak. Two agents for a job one can handle just doubles the cost. Add specialists only when sub-tasks need genuinely different tools or reasoning.

M15 Code Interpreter & Sandbox**CORE IDEA**

Claude writes code. A sandbox runs it.

KEY PSEUDOCODE

```

DEFINE tool "run_python":
  input: code (string)
  output: { stdout, stderr, exit_code, time }

FUNCTION run_python(code):
  sandbox = SPAWN ephemeral container
  # --network=none --memory=256m
  # --cpus=1 --read-only
  # --timeout=30s

  result = sandbox.EXEC(code)
  DESTROY sandbox # always
  RETURN result # structured, never raw

# Agent loop:

```

REMEMBER

✗ *"Claude runs the code on Anthropic's servers."*

✓ No. Claude only generates the code as text. You provide the sandbox — a Docker container on your machine, an E2B cloud sandbox you signed up for, or a WASM runtime in the browser. Anthropic never executes user code on Claude's behalf; that's your responsibility.

M15B Build an Agent + Subagent System**CORE IDEA**

One agent can't be a generalist forever.

KEY PSEUDOCODE

```

# Subagent spec lives at .claude/agents/research.md
# role: filing search specialist | tools: [search, get_details]

FUNCTION run_subagent(spec, task_prompt):
  # Fresh, isolated context every call
  context = [system: spec.role, user: task_prompt]
  LOOP:
    reply = CALL Claude(context, tools=spec.tools)
    IF reply.stop_reason == "tool_use":
      run tool, append result, continue
    ELSE: RETURN parse_json(reply.text)

FUNCTION coordinator(user_question, history):
  task1 = build_task("research", user_question, history)

```

REMEMBER

✗ *"Subagents are just sub-prompts — same context, different instructions."*

✓ No. A subagent is a fresh agent invocation with its own system prompt, tools, and a blank context window. It does not see the coordinator's conversation. If the coordinator doesn't pass a value in the task prompt, the subagent literally cannot know it.

M16 Input Guardrails**CORE IDEA**

Never let raw input touch the model.

KEY PSEUDOCODE

```
FUNCTION guard(user_input, user_id):
  # 1. Rate limit (cheapest, run first)
  IF NOT rate_bucket[user_id].consume(1):
    RETURN block("rate_limit")

  # 2. PII scan + redact (sub-ms regex)
  found = scan_pii(user_input) # SSN, CC, email, phone
  clean = redact(user_input, found) # → [REDACTED_SSN]

  # 3. Length / structure / schema
  IF len(clean) > MAX OR NOT schema.validate(clean):
    RETURN block("malformed")

  # 4. Injection + jailbreak classifier (separate Claude call)
  verdict = classify(clean) # safe / suspicious / malicious
```

REMEMBER

✗ *"Claude is the guardrail — just tell it 'don't do bad things' in the system prompt."*

✓ System-prompt rules are advisory, not enforced. A clever injection can talk Claude out of them. Real guardrails run outside the model — programmatic checks, separate classifier calls, schema validators — that the user input physically cannot influence.

M17 Output Guardrails & HITL**CORE IDEA**

Never let raw output reach the user.

KEY PSEUDOCODE

```
FUNCTION guard_output(claude_response, tool_calls, sources):
  # 1. Output PII scan + redact
  found = scan_pii(claude_response)
  clean = redact(claude_response, found) # → [REDACTED_SSN]

  # 2. Content / policy classifier (separate Claude call)
  verdict = classify_content(clean)
  IF verdict == "unsafe": RETURN block("policy")

  # 3. Groundedness check vs retrieved sources
  claims = verify_claims(clean, sources)
  IF any(c.status == "contradicted" FOR c IN claims):
    RETURN block("hallucination")

  # 4. Risk-score every proposed tool call
```

REMEMBER

✗ *"Input guardrails are enough — if I sanitise the input, the output is safe."*

✓ No. Output can still leak PII it pulled from your RAG corpus, hallucinate facts the user never asked about, or propose a tool call that wasn't in the original prompt at all. Output is a separate attack surface with its own failure modes (hallucination, data leakage, dangerous tool use). M16 + M17 are two halves of the sandwich, not redundant layers.

M18 Evaluation & Testing**CORE IDEA**

Test what's correct, not what's identical.

KEY PSEUDOCODE

```
FUNCTION run_eval(agent_version, golden_dataset, rubric):
    results = []
    FOR EACH (input, expected_behavior) IN golden_dataset:
        actual_output = agent_version.run(input) # capture trace too
        score = llm_judge(input, actual_output, rubric) # separate Claude call
        results.append({input, output: actual_output, score, expected: expected_behavior})

    pass_rate = COUNT(s.passed for s in results) / LEN(results)
    per_criterion = {crit: AVG(s[crit] for s in results) for crit in rubric}

    STORE(agent_version, pass_rate, per_criterion, timestamp)
    IF pass_rate < BASELINE - 0.02: # regression threshold (2pp)
        RETURN block_deploy("regression detected")
    RETURN allow_deploy(pass_rate, per_criterion)
```

REMEMBER

✗ *"I'll write unit tests with assertEquals(output, expected)."*

✓ No — LLM outputs vary every run, even at temperature zero across model versions. You need rubric-based grading on sampled outputs, not exact-match assertions on a single output. Test for properties (cites a source, answers the question, valid JSON), not for strings.

M19 Tracing & Logging**CORE IDEA**

"It broke" tells you nothing. A trace tells you everything.

KEY PSEUDOCODE

```
FUNCTION handle_request(user_input):
    trace_id = NEW_UUID()
    WITH span("agent.run", trace_id, parent=NULL) AS root:
        root.attrs = {user_id, input_len: len(user_input)}

    WHILE NOT done:
        WITH span("llm.call", trace_id, parent=root) AS s:
            response = claude.messages.create(...)
            s.attrs = {model, tokens_in, tokens_out, cost}

    IF response.has_tool_call:
        WITH span("tool." + name, trace_id, parent=root) AS t:
            result = run_tool(name, args)
            t.attrs = {args, result_size, latency_ms}
```

REMEMBER

✗ *"I have logs already — print(response) is enough."*

✓ Print statements have no trace_id, no parent relationship, no structured fields, no queryability. Restoring causality from print logs at 3am is what kills startups. Structured spans solve this from day one — and they cost nothing extra to set up if you do it before the first 1,000 requests.

CORE IDEA

Tracing captures data. Monitoring turns it into decisions.

KEY PSEUDOCODE

```
# DASHBOARD QUERY (runs continuously)
SELECT model, tool, p95_latency, error_rate, avg_cost
FROM traces
WHERE timestamp > NOW() - 1hour
GROUP BY model, tool

# ALERT RULE (runs every minute)
IF current.error_rate > baseline.error_rate * 1.5:
  PAGE on_call("error rate regression in tool X")

# FEEDBACK LOOP
ON user_feedback(trace_id, rating):
  STORE feedback
  IF rating == thumbs_down:
```

REMEMBER

✗ *"I'm tracking latency and error rate — that's enough monitoring."*

✓ Latency tells you it's slow; it doesn't tell you it's wrong. An agent can be fast, cheap, error-free, and still produce hallucinated answers. Add task-completion rate, satisfaction, and content-quality signals from M18 evals running on production traffic samples.

M21 API Design & Deployment**CORE IDEA**

Your agent isn't a product until it's an API.

KEY PSEUDOCODE

```
ENDPOINT POST /v1/agent/run:
# 1. Auth
api_key = REQUEST.header("Authorization")
user = AUTHENTICATE(api_key) # 401 if invalid

# 2. Rate limit (per-key token bucket)
IF NOT bucket[user.id].consume(1):
RETURN 429 "rate_limited"

# 3. Idempotency cache
idem_key = REQUEST.header("Idempotency-Key")
IF cached := cache.get(idem_key):
RETURN cached # safe replay

# 4. Validate body, dispatch to right lane
```

REMEMBER

✗ *"FastAPI/Express + a Claude SDK call = production API."*

✓ That's a demo. Production needs auth, rate limiting, idempotency, versioning, observability, autoscaling, graceful timeouts, and a queue for long jobs. The framework is 5% of the work — the surrounding plumbing is what keeps it up at 3am.

M22 Cost Optimization**CORE IDEA**

Cost is a design problem, not a billing problem.

KEY PSEUDOCODE

```
FUNCTION run_agent(user_input):
# 1. Cache lookup — full response cache (semantic or exact)
IF cached := response_cache.get(user_input):
RETURN cached # 100% savings

# 2. Route — pick the cheapest capable model
task_class = router_classify(user_input) # cheap Haiku call
model = {trivial: HAIKU, normal: SONNET, hard: OPUS}[task_class]

# 3. Compress — trim context to what's actually needed
context = compress(history, recent=5_turns, summarize_older=TRUE)

# 4. Prompt cache — mark stable prefix as cacheable
msg = build_message(
system=SYSTEM_PROMPT, # cache_control: ephemeral
```

REMEMBER

✗ *"Just use Haiku everywhere — it's the cheapest."*

✓ Haiku is great for classification and short-form generation, but stretching it to do reasoning means more retries, more iteration loops, and more total tokens — with worse quality. Cost = price-per-token × tokens-needed. Right-sized model wins on both axes.

CORE IDEA

One agent, three runtimes — portable by design.

KEY PSEUDOCODE

```
# SHARED – the agent core (never changes per runtime)
FUNCTION agent_core(question):
  msgs = [{role: "user", content: question}]
  WHILE step < max_steps:
    resp = claude.messages.create(model, tools, msgs)
    IF resp.stop_reason == "end_turn": RETURN resp.text
    msgs += run_tools(resp.tool_calls) # tool loop

# ADAPTER 1 – Local Docker (FastAPI + uvicorn)
ROUTE POST /query AS docker_main(req):
  RETURN {answer: agent_core(req.question)} # long-lived process

# ADAPTER 2 – GCP Cloud Run (same FastAPI image)
# gcloud run deploy --image=REGISTRY/agent --memory=1Gi
```

REMEMBER

✗ *"Lambda is always the cheapest option."*

✓ Not for agents. A single run can burn 10–30 seconds of compute (multiple Claude calls + tools), and Lambda bills per millisecond plus cold starts. Cloud Run with min-instances=0 is often cheaper for steady traffic.

M23 Capstone Project Series**CORE IDEA**

Capstones are the integration test for everything you learned.

KEY PSEUDOCODE

```
FUNCTION capstone_cycle(n, domain):
  # 1. Pick or carry forward your domain anchor
  domain = pick_once("A_healthcare", "B_ecom", "C_ucc")

  # 2. Read the spec for capstone N (it builds on N-1)
  spec = read("capstones/C" + n + "/spec/agent-spec.md")
  carry = load_artifacts_from(prior_capstones=[1..n-1])

  # 3. Let Claude generate the project from spec + carry-over
  project = claude.generate(spec, context=carry, sdk="claude-agent-sdk")

  # 4. You verify against the rubric (functionality, quality,
  # prompts, safety, observability) and run the eval harness
  score = run_evals(project) + self_assess(project, rubric)

  # 5. If score < bar, iterate on the SPEC (not the code)
```

REMEMBER

✗ *"Capstones are optional polish — the modules are the real course."*

✓ Backwards. Modules teach skills in isolation; capstones are where you find out whether you can integrate them. Skipping them is like a pilot skipping simulator hours — you can recite the checklist but you have not flown the plane.

M24 What's Next — The Agent Frontier**CORE IDEA**

The model improves on its own. Architecture is what compounds.

KEY PSEUDOCODE

```
FUNCTION autonomous_agent(goal, deadline="7 days"):
  memory = load_persistent_memory(user_id) # survives sessions
  plan = orchestrator.decompose(goal, memory) # multi-step plan

  FOR step IN plan:
    specialist = route_to_subagent(step.kind) # researcher / coder / reviewer
    tools = [computer_use, browser, code_sandbox, search]

    TRY:
      result = specialist.run(step, tools, budget=step.budget)
      eval_score = evals.judge(result, step.criteria) # M18 patterns
      IF eval_score < threshold: request_human_review(step, result)
    CHECKPOINT(step, result) # resumable across days

  EXCEPT any_error:
```

REMEMBER

✗ *"The model will get smart enough that I won't need architecture."*

✓ Better models raise the ceiling; architecture is the floor. A smarter model with no guardrails or evals fails just as silently — just more confidently. Capability gains amplify the cost of bad scaffolding, they don't excuse it.

M25 Claude Code Mastery**CORE IDEA**

Claude Code is an agent — configure it like a teammate.

KEY PSEUDOCODE

```
# Project layout
my-repo/
├─ CLAUDE.md # conventions, gotchas, key files
├─ .claude/
│   ├─ commands/release.md # /release slash command
│   ├─ agents/security.md # subagent: security reviewer
│   ├─ mcp.json # MCP server config (GitHub, etc)
│   └─ settings.json # permissions + hooks

# commands/release.md frontmatter
---
description: "Cut a release: bump version, tag, push"
allowed-tools: [Bash(git:*), Bash(npm:*), Read]
```

REMEMBER

✗ *"Claude Code is just an IDE chatbot."*

✓ It's a full agent: ReAct loop, tool use, file edits, shell, search, MCP. Same architecture as the production agents you learned to build — running locally over your codebase.

M26 Hooks, Sessions, Agent SDK**CORE IDEA**

Control plane, state plane, build plane.

KEY PSEUDOCODE

```
# 1. CONTROL PLANE — .claude/settings.json (hook config)
{
  "hooks": {
    "PreToolUse": [{
      "matcher": "Bash",
      "hooks": [{ "command": "./scripts/policy_check.sh" }]
    }] # exit 2 from script = BLOCK the tool
  }
}

# 2. STATE PLANE — resume yesterday's conversation
$ claude --resume session_id # full context loads from disk

# 3. BUILD PLANE — Agent SDK with hooks injected
```

REMEMBER

✗ *"Hooks are nice-to-have logging."*

✓ Hooks are a control plane, not a log shim. A PreToolUse hook returning exit code 2 blocks the tool call before it runs — the model has to choose differently. That's policy enforcement, secret redaction, and approval gates — the same mechanism the cert exam asks about.

CORE IDEA

The exam tests reasoning, not recall.

KEY PSEUDOCODE

```
# ===== 4-WEEK STUDY PLAN =====  
FOR week IN [1, 2, 3, 4]:  
  IF week == 1: domains = [D1_Foundations, D2_Tools]  
  IF week == 2: domains = [D3_RAG_Memory, D4_MultiAgent]  
  IF week == 3: domains = [D5_Production] + weakest_from_w1_w2  
  IF week == 4: domains = ["full mocks", "sleep"]  
  
  FOR d IN domains:  
    review_scenarios(d, count=5) # pattern library  
    drill_questions(d, count=20) # elimination reps  
    log_misses(d) → weak_domain_list  
  
# ===== IN-EXAM ANSWER DECISION TREE =====  
FOR q IN exam_questions:
```

REMEMBER

✗ *"If I read the docs cover-to-cover, I'll pass."*

✓ Doc reading builds recall; the exam tests reasoning. You can know every method and still fail because you've never decided which pattern fits which scenario. Replace doc reading with scenario drills as soon as you've covered the basics.

Quick Reference Cards

The Tool Use Loop

```
messages = [user question]
WHILE true:
    response = claude(messages, tools)
    IF stop_reason == "end_turn": RETURN response
    FOR each tool_use in response:
        result = EXECUTE tool
        APPEND tool_result to messages
```

The 8 Design Patterns

- | | |
|----------------------------|--|
| 1. Single-Turn | one tool, one call |
| 2. ReAct | think → act → observe → loop |
| 3. Plan-Execute | decompose first, then run |
| 4. Router | classify input, route to handler |
| 5. Parallel Fan-Out | same task × many inputs |
| 6. Pipeline | sequential stages |
| 7. Supervisor | coordinator + specialist workers |
| 8. Autonomous+HITL | agent runs, human approves key decisions |

The 3 Agent Approaches

- | | |
|-------------------|--|
| Raw (M15B) | ~250 lines, full control, write every line |
| SDK (M26) | ~40 lines, hooks + sessions, SDK runs the loop |
| Spec (M25) | ~100 lines of spec, Claude Code generates everything |

Cost Cheat Sheet

- | | |
|---------------|---|
| Haiku | \$0.25/1M in, \$1.25/1M out — simple tasks |
| Sonnet | \$3/1M in, \$15/1M out — most agent work |
| Opus | \$15/1M in, \$75/1M out — complex reasoning |
| Cache | 90% savings on repeated system prompts |
| Batch | 50% discount for non-real-time |

The 3 Deployment Tiers

- | | |
|---------------|---|
| Tier 1 | Docker + DuckDB (free, local) |
| Tier 2 | GCP Cloud Run + BigQuery (pay-per-use) |
| Tier 3 | AWS Lambda + API Gateway (event-driven) |